

Definiții Curs MAP

Java

I. Concepte OOP de bază

A. Clase și obiecte

1. **Clasa** reprezintă un șablon care definește structura (atribute) și comportamentul (metode) obiectelor.
2. **Obiectul** este o instanță a unei clase care este creată la execuție și alocată pe heap.

B. **Încapsularea** este separarea reprezentării interne a datelor de interfața externă. Se realizează prin modificatori de acces precum: private, protected, public.

C. **Moștenirea** este mecanismul prin care o clasă preia atributele și metodele unei alte clase, permițând reutilizarea codului și crearea ierarhiilor.

D. **Polimorfismul** este capacitatea unei entități de a reacționa diferit în funcție de context.

E. **Abstractizarea** este procesul prin care se expun doar caracteristicile esențiale ale unui obiect, ascunzând detaliile complexe de implementare. (Un exemplu concret ar fi dacă ne-am gândi la o telecomandă. Ele pot fi de la Sony, Samsung etc. Fiecare are funcțiile ei, dar în general toate au un buton de On / Off, de volum, de schimbat canalul, etc. Astfel, această telecomandă ”generală” poate fi considerată o abstracție)

F. Overloading vs Overriding

1. **Overloading** se referă la definirea mai multor metode cu același nume în aceeași clasă, dar cu semnături diferite: număr / tip diferit de parametri.
2. **Overriding** se referă la modificarea implementării unei metode moștenite din clasa părinte în clasa copil, păstrând aceeași semnătură.

G. Legare statică vs dinamică

1. **Legare statică:** Asocierea metodei se face la compilare (ex. metode statice, private, final, overloading).
2. **Legare dinamică:** Asocierea metodei se face la rulare, în funcție de tipul obiectului (ex. overriding)

H. **Clasele abstracte** sunt clasele declarate cu abstract care nu pot fi instanțiate. Pot conține metode abstracte și metode concrete.

I. **Interfețele** sunt un ”contract” la care aderă fiecare clasă care implementează interfața respectivă. Ele conțin metode abstracte (implicit publice), metode default, statice și constante.

J. Principii de proiectare:

1. SOLID

- a) **Single Responsibility:** Fiecare clasă are o singură responsabilitate

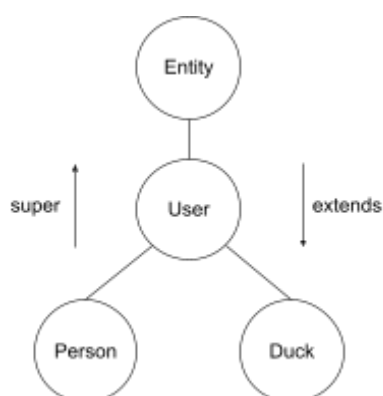
- b) **Open-Closed**: Entitățile ar trebui să fie deschise pentru extindere, dar închise pentru modificare
 - c) **Liskov Substitution**: Orice clasă copil ar trebui să poată înlocui clasa părinte fără a strica integritatea aplicației (Poate vă ajută să rețineți mai ușor uitându-vă la cuvântul cheie "Substitution", substituie, care duce la înlocuire și mai apoi la definiție)
 - d) **Interface Segregation**: În loc de utilizarea unei interfețe cu multe metode, din care doar câteva sunt folosite, mai bine avem mai multe interfețe mai mici și le folosim doar pe cele necesare.
 - e) **Dependency Inversion**: Dacă o clasă are o dependență, această dependență ar trebui să fie o interfață, nu o clasă concretă.
2. **Low Coupling**: Dependențe minime între clase. (Clasele cu cât sunt mai independente și mai puțin întortocheate în relații, cu atât e mai ușor să fie testate și înțelese)
 3. **High Cohesion**: O clasă trebuie să aibă responsabilități strâns legate între ele. (Are legătură cu principiul singurei responsabilități și presupune ca o funcție să nu facă lucruri diverse. Un exemplu absurd ar fi să nu calculezi distanța unei satelit până la Pământ și timpul de așteptare până ajunge comanda de pe Glovo în aceeași clasă. Faci alta separat)

K. **Contractele între obiecte** se referă în principal la utilizarea interfețelor sau a metodelor publice care stabilesc ce face un obiect, fără a expune cum face.

II. Genericitate

- A. **Tipurile generice** permit definirea claselor, interfețelor și metodelor cu parametri de tip (ex. List<E>, Class<T>), asigurând reutilizarea codului și siguranța tipurilor la compilare
- B. **Metodele genetice** introduc proprii parametri de tip, independenți de clasa în care se află (ex. public <T> void swap(T[] a, int i, int j)).
- C. **Constrângerile pe tipurile generice** restricționează tipurile ce pot fi folosite ca argumente (ex. <T extends Number> - T trebuie să fie număr sau subclasă)
- D. **Wildcards** (Simbolul ? pentru a reprezenta un tip necunoscut)
 1. ? extends T (Limita de sus): Acceptă T sau subclase (read-only safe)
 2. ? super T (Limita de jos): Acceptă T sau superclase (write safe)

Ex. La laborator am folosit ierarhia Entity, User, Person, Duck, conform imaginii de mai jos. Când specificăm limita superioară, de exemplu List<? extends User>, noi putem avea în listă orice de la User în jos (User, Person, Duck). Dacă specificăm limita inferioară, de exemplu List<? super User>, noi putem avea în listă doar de la User în sus (User, Entity). Arborele trebuie vizualizat de jos în sus.



- E. **Colecții generice**: Structuri de date din Java Collections Framework (List<E>, Set<E>, Map<K, V>) care stochează elemente de un anumit tip specificat.

III. Excepții

A. Checked vs Unchecked exceptions:

1. **Checked** (Exception): Excepții verificate la compilare. Programatorul este obligat să le trateze (try-catch) sau să le declare (throws). Ex: IOException, SQLException.
2. **Unchecked** (RuntimeException): Excepții neverificate la compilare, indică erori de programare. Nu este obligatorie tratarea lor. Ex: NullPointerException, IndexOutOfBoundsException.

B. Try-catch-finally:

1. **try**: Blocul de cod unde poate apărea excepția.
2. **catch**: Blocul care prinde și tratează excepția.
3. **finally**: Blocul care se execută întotdeauna (indiferent dacă a apărut sau nu o excepție), util pentru închiderea resurselor.

C. Excepții custom: Crearea propriilor tipuri de excepții prin extinderea clasei Exception (pentru checked) sau RuntimeException (pentru unchecked).

IV. Interfețe funcționale / Operații pe stream-uri

A. Metode default și static în interfețe: Permit adăugarea de implementări în interfețe fără a sparge compatibilitatea cu clasele existente.

B. Interfețe funcționale: Interfețe care au exact o metodă abstractă. Pot fi marcate cu @FunctionalInterface. Ex: Runnable, Comparator, Predicate.

C. Lambda expressions: Funcții anonime (fără nume) care pot fi tratate ca date. Sintaxa: (parametri) -> { corp }. Implementează concis o interfață funcțională.

D. Referințe la metode: Sintaxă prescurtată pentru lambda care doar apelează o metodă existentă (ClassName::methodName).

E. Tipuri de interfețe funcționale:

1. **Predicate<T>**: boolean test(T t) - verifică o condiție
2. **Consumer<T>**: void accept(T t) - consumă o valoare (ex. afișare)
3. **Function<T, R>**: R apply(T t) - transformă un input T într-un output R
4. **Supplier<T>**: T get() - furnizează o valoare
5. **Comparator<T>**: int compare(T o1, T o2) - compară două obiecte

F. Optional: Un container (Optional<T>) care poate conține o valoare sau null, prevenind NullPointerException. Metode: isPresent(), ifPresent(), orElse().

G. Stream API: O secvență de elemente care suportă operații secvențiale și paralele de procesare a datelor. Nu stochează date, doar le procesează.

1. Operații intermediare (lazy):

- a) **filter**: Filtrează elementele pe baza unui Predicate.
- b) **map**: Transformă elementele folosind o Function.
- c) **sorted**: Sortează elementele (natural sau cu Comparator).

2. Operații terminale (eager - declanșează procesarea):

- a) **forEach**: Iterează elementele (Consumer).

- b) **collect**: Transformă stream-ul într-o colecție (toList, toSet, toMap).
- c) **groupingBy**: Grupează elementele pe baza unei chei (în collect).
- d) **reduce**: Combină elementele într-o singură valoare (acumulare).
- e) **anyMatch**, **allMatch**, **noneMatch**: Verifică dacă elementele satisfac un predicat (returnează boolean).
- f) **count**: Numără elementele.

V. Concurență și Multithreading

- A. **Thread-uri**: Cea mai mică unitate de procesare planificabilă de către sistemul de operare. Java oferă clasa Thread. Poate fi găsit în 6 stadii:
 1. New - Thread-ul a fost creat, dar nu a început execuția
 2. Running - Thread-ul este pregătit să ruleze sau rulează
 3. Blocked - Thread-ul așteaptă după o resursă
 4. Waiting - Thread-ul așteaptă după alt thread
 5. Timed Waiting - Thread-ul așteaptă un anumit timp fix
 6. Terminated - Thread-ul și-a terminat execuția
- B. **Runnable**: Interfață funcțională (void run()) ce definește o sarcină de executat pe un thread.
- C. **Callable**: Similar cu Runnable, dar poate returna un rezultat (V call()) și poate arunca excepții.
- D. **Future**: Reprezintă rezultatul asincron al unei operații Callable. Permite verificarea stării și obținerea rezultatului (get()).
- E. **ExecutorService**: Un framework pentru gestionarea execuției asincrone, decuplând crearea thread-urilor de logica aplicației.
- F. **Thread pool**: Un set de thread-uri reutilizabile gestionate de un Executor. Reduce costul creării repetate a thread-urilor.
- G. **Programare asincronă**: Model de execuție în care operațiile nu blochează firul principal de execuție, rezultatele fiind disponibile ulterior (prin Future / Callbacks).

C#

I. Introducere în C#

- A. **Arhitectura .NET**:
 1. CLR (Common Language Runtime): Mediul de execuție care gestionează memoria (Garbage Collection), thread-urile și securitatea. Codul C# compilează în CIL (Common Intermediate Language), care este apoi compilat JIT (Just-In-Time) în cod mașină nativ.
- B. Value Types vs. Reference Types:
 1. **Value Types** (struct, enum, primitive): Stocate pe Stack. Conțin direct datele. La atribuire (a = b), se copiază valoarea (modificarea lui a nu afectează b). Nu pot fi null (decât dacă sunt Nullable).

2. **Reference Types** (class, interface, delegate, string, array): Stocate pe Heap. Variabila conține doar o adresă de memorie. La atribuire, se copiază referința (ambele variabile indică spre același obiect).
- C. **Struct vs Class**: În C#, struct este un tip valoare (folosit pentru obiecte mici, imutabile), în timp ce class este tip referință. Java nu are structuri definite de utilizator (doar clase).
- D. **Proprietăți (Properties)**:
 1. Înlocuiesc getterii / setterii expliți din Java, oferind o sintaxă simplificată pentru accesul la date.
 2. Auto-implemented properties: `public int Age { get; set; }` (compilatorul generează automat câmpul privat).
- E. Modificatori de parametri:
 1. **ref**: Transmite variabila prin referință. Variabila trebuie inițializată înainte de apel. Modificările din metodă se reflectă în exterior.
 2. **out**: Transmite variabila prin referință, dar nu necesită inițializare înainte. Metoda apelată este obligată să îi atribuie o valoare.
 3. **params**: Permite un număr variabil de argumente (similar cu varargs din Java), dar trebuie să fie ultimul parametru.
- F. **Nullable Types**: Tipurile valoare nu pot fi null implicit. Se folosește sintaxa `T?` (ex: `int?`, `bool?`) care este un shorthand pentru `Nullable<T>`. Util pentru baze de date.
- G. Operatori specifici:
 1. **as**: Încearcă conversia (cast); dacă eșuează, returnează null (nu aruncă excepție).
 2. **is**: Verifică compatibilitatea tipului (`instanceof`).
 3. **?? (Null-coalescing)**: `a ?? b` returnează `a` dacă nu e null, altfel `b`.
- H. **Tablouri (Arrays)**: C# diferențiază între Tablouri multidimensionale (`int[,]`, matrice dreptunghiulară, stocare contiguă) și Tablouri de tablouri (`int[][]`, "jagged arrays", vector de vectori, similar Java).
- I. Genericitate:
 1. **Reified Generics**: Tipurile generice există la runtime în C#. Nu există Type Erasure ca în Java. Consecință: `List<int>` și `List<string>` sunt tipuri complet diferite la execuție. Performanță mai bună pentru tipuri valoare (fără boxing/unboxing excesiv).
 2. **Constrângeri (Constraints)**: Mai flexibile, folosind clauza `where`. `where T : struct` (tip valoare), `where T : class` (tip referință), `where T : new()` (constructor fără parametri).
- J. **Delegați (Delegates)**:
 1. Un tip referință care definește o semnătură de metodă (pointer la funcție type-safe).
 2. Multicast: Pot stoca referințe către mai multe metode, apelate în lanț (operatorii `+=`, `-=`).

K. Evenimente (Events):

1. O abstractizare peste delegați, implementând pattern-ul Publisher-Subscriber.
2. Restricționează accesul: doar clasa care declară evenimentul îl poate invoca (fire), clienții se pot doar abona/dezabona (+=, -=).

L. Extension Methods:

1. Permit adăugarea de metode noi unor tipuri existente (chiar și sealed sau primitive) fără moștenire.
2. Se definesc ca metode statice în clase statice, iar primul parametru are modificatorul this (ex: public static void Metoda(this String s)). Stau la baza LINQ.

M. LINQ (Language Integrated Query):

1. Sintaxe
 - a) **Query Syntax**: Stil declarativ SQL (from x in list where x > 5 select x).
 - b) **Method Syntax**: Fluent API folosind metode de extensie și lambda (list.Where(x => x > 5).Select(x => x)).
2. Mecanism de Execuție:
 - a) **Deferred Execution** (Amânată): Interogarea nu se execută la definire. Se execută doar când se iterează rezultatul (ex: foreach). Metode: Where, Select, Take.
 - b) **Immediate Execution** (Imediată): Interogarea se execută pe loc și returnează datele. Metode care "materializează" rezultatul: ToList(), ToArray(), Count(), First().